

گزارش سؤال دوم تمرین اول

بررسی اثر بهمنی در DES استاندارد و DES با S-box تصادفی

محمد امانلو - شماره دانشجویی: ۸۱۰۱۰۰۰۸۴

۱ مقدمه

در این بخش از تمرین، من پیاده‌سازی سؤال اول را مبنا قرار دادم و آن را طوری گسترش دادم که بتوانم avalanche effect یا همان اثر بهمنی را اندازه‌گیری کنم. ایده اصلی سؤال این بود که یک بار همین DES استاندارد را با جدول‌های اصلی S-box اجرا کنم و یک بار هم همان ساختار را نگه دارم ولی به جای S-box های استاندارد، از S-box های تصادفی استفاده کنم. بعد اثر بهمنی هر دو حالت را حساب کنم و ببینم رفتارشان چه فرقی با هم دارد.

من سعی کردم هم از نظر پیاده‌سازی، که تمیز و قابل فهمی بنویسم و هم از نظر مفهومی دقیق بفهمم که اثر بهمنی دقیقاً چیست، چرا مهم است، و چرا S-box ها در این موضوع نقش خیلی مهمی دارند. در واقع این سؤال فقط یک تست ساده روی خروجی نبود؛ بیشتر هدفش این بود که ببینم آیا ساختار DES واقعاً خاصیت پخش‌شدگی و آشفتگی لازم را دارد یا نه، و اگر جدول‌های غیرمهندسی شده و تصادفی را جایگزین کنیم، نتیجه چه تغییری می‌کند.

۲ صورت مسئله و برداشت من از خواسته سؤال

برداشت من از سؤال این بود که باید چند کار مشخص انجام بدهم:

۱. پیاده‌سازی سؤال اول را نگه دارم، یعنی همان DES استاندارد که با S-box های اصلی کار می‌کند.
۲. یک نسخه دیگر از همان DES بسازم که فقط در بخش S-box فرق داشته باشد، یعنی به جای جدول‌های استاندارد، از جدول‌های تصادفی استفاده کند.
۳. این جدول‌های تصادفی را با ابعاد درست بسازم؛ یعنی برای هر S-box یک جدول 16×4 داشته باشم و در مجموع ۸ تا از این جدول‌ها تولید کنم.
۴. مقدارهای داخل هر خانه را به صورت تصادفی ولی در بازه درست، یعنی بین ۰ تا ۱۵، تولید کنم.
۵. برای هر نسخه، اثر بهمنی را حساب کنم. منظور از اثر بهمنی در اینجا این است که یک بیت از ورودی را عوض کنم و ببینم به طور متوسط چند بیت از خروجی عوض می‌شود.

۶. نتایج به دست آمده را با هم مقایسه کنیم و تحلیل کنیم که DES استاندارد و DES با S-box تصادفی چه رفتاری نشان می دهند.

پس در عمل، من یک موتور DES نوشتم که بتواند با دو نوع S-box کار کند: یکی استاندارد و یکی تصادفی. بعد روی ورودی های مختلف این مقایسه را انجام دادم.

۳ مرور تئوری DES

۱.۳ ایده کلی DES

DES یک الگوریتم رمزنگاری بلوکی متقارن است. یعنی هم برای رمزگذاری و هم برای رمزگشایی از یک کلید پایه استفاده می شود. این الگوریتم روی بلوک های ۶۴ بیتی کار می کند و ساختارش از نوع Feistel Network است. در DES داده ورودی ابتدا یک جایگشت اولیه می خورد، بعد وارد ۱۶ دور پردازش می شود، و در انتها هم یک جایگشت نهایی روی آن انجام می شود. نکته مهم DES این است که امنیتش فقط از یک عمل ساده مثل XOR به دست نمی آید. چیزی که آن را جالب می کند ترکیب چند ایده با هم است: گسترش بیت ها، XOR با کلید دور، عبور از S-box ها، و در نهایت جایگشت. همین ترکیب است که باعث می شود رابطه ساده ای بین ورودی و خروجی وجود نداشته باشد.

۲.۳ ساختار Feistel

در ساختار Feistel، بلوک ۶۴ بیتی بعد از جایگشت اولیه به دو نیمه ۳۲ بیتی تقسیم می شود: نیمه چپ و نیمه راست. در هر دور، نصف سمت راست وارد تابع F می شود و خروجی این تابع با نصف سمت چپ XOR می شود. بعد نقش دو نیمه جابه جا می شود. اگر بخواهم خیلی ساده بگویم، در هر دور این اتفاق می افتد:

$$L_i = R_{i-1}$$

$$R_i = L_{i-1} \oplus F(R_{i-1}, K_i)$$

که در آن:

- L_i نیمه چپ در دور i است
- R_i نیمه راست در دور i است
- K_i کلید دور i است

قشنگی ساختار Feistel این است که با همین ساختار می شود هم رمزگذاری کرد و هم رمزگشایی؛ فقط کافی است ترتیب کلیدهای دور برعکس شود. البته در این سؤال فقط روی رمزگذاری و تحلیل اثر بهیمنی تمرکز داشتم.

۳.۳ تابع F در DES

تابع F قلب DES است. این تابع ورودی ۳۲ بیتی را می‌گیرد و با کمک کلید ۴۸ بیتی همان دور، یک خروجی ۳۲ بیتی می‌سازد. روند کار تابع F به صورت خلاصه این است:

۱. ابتدا نیمه راست ۳۲ بیتی را با جدول Expansion به ۴۸ بیت تبدیل می‌کنیم.

۲. نتیجه را با کلید دور XOR می‌کنیم.

۳. خروجی ۴۸ بیتی را به ۸ قسمت ۶ بیتی تقسیم می‌کنیم.

۴. هر قسمت ۶ بیتی وارد یکی از ۸ عدد S-box می‌شود و به ۴ بیت تبدیل می‌شود.

۵. ۸ خروجی ۴ بیتی کنار هم قرار می‌گیرند و یک بلوک ۳۲ بیتی تشکیل می‌دهند.

۶. روی این بلوک ۳۲ بیتی، جایگشت P اعمال می‌شود.

بخشی که از نظر غیرخطی بودن خیلی مهم است همان S-box ها هستند. اگر S-box ها نباشند، DES عملاً ساختار خیلی ضعیف‌تری پیدا می‌کند.

۴ نقش S-box در DES

۱.۴ چرا S-box مهم است؟

S-box ها جایی هستند که DES از یک سیستم صرفاً خطی خارج می‌شود. عمل‌هایی مثل XOR و جایگشت به تنهایی خطی‌اند، اما S-box ها یک نگاهت غیرخطی ایجاد می‌کنند. همین غیرخطی بودن است که باعث می‌شود تحلیل الگوریتم برای مهاجم سخت‌تر شود.

در DES، هر S-box یک ورودی ۶ بیتی را می‌گیرد و یک خروجی ۴ بیتی می‌دهد. برای تعیین خروجی، دو بیت اول و آخر ورودی، شماره سطر را تعیین می‌کنند و چهار بیت وسط، شماره ستون را. بعد مقدار داخل خانه متناظر از جدول خوانده می‌شود.

۲.۴ چرا در این سؤال S-box تصادفی ساختیم؟

هدف این سؤال این بود که ببینیم اگر به جای S-box های استاندارد که با دقت طراحی شده‌اند، جدول‌هایی با اعداد تصادفی بگذاریم، اثر بهیمنی چه تغییری می‌کند. این موضوع مهم است چون نشان می‌دهد فقط داشتن یک جدول تصادفی کافی نیست؛ خیلی وقت‌ها جدول‌های استاندارد به خاطر ملاحظات امنیتی، مهندسی شده طراحی می‌شوند و ویژگی‌هایی دارند که از بیرون خیلی واضح نیست.

۵ اثر بهمنی چیست؟

۱۰۵ تعریف اثر بهمنی

اثر بهمنی یعنی اگر یک تغییر خیلی کوچک در ورودی یا کلید ایجاد کنیم، خروجی باید تغییر زیادی بکند. در حالت ایده‌آل، اگر یک بیت از ورودی را عوض کنیم، تقریباً نصف بیت‌های خروجی باید عوض شوند. چون در این سؤال خروجی ۶۴ بیت است، انتظار شهودی این است که اگر الگوریتم رفتار خوبی داشته باشد، با برعکس کردن یک بیت از ورودی، به‌طور متوسط چیزی نزدیک به ۳۲ بیت از خروجی عوض شود.

۲۰۵ چرا اثر بهمنی مهم است؟

اگر یک الگوریتم اثر بهمنی خوبی نداشته باشد، تغییرهای کوچکی در ورودی ممکن است فقط روی بخش کوچکی از خروجی اثر بگذارند. این حالت خطرناک است چون بین ورودی و خروجی یک رابطه قابل ردگیری تر ایجاد می‌شود. اما اگر یک تغییر تک بیتی باعث آشفتگی گسترده در خروجی شود، تحلیل الگوریتم سخت‌تر می‌شود.

این مفهوم از دو ایده مهم در رمزنگاری هم پشتیبانی می‌کند:

- **Dffusion** یا پخش‌شدگی: یعنی اطلاعات هر بیت از ورودی در بخش بزرگی از خروجی پخش شود.

- **Confusion** یا آشفتگی: یعنی رابطه بین کلید، ورودی و خروجی پیچیده و غیرمستقیم باشد.

DES با ترکیب ساختار Feistel، جایگشت‌ها و S-boxها تلاش می‌کند هر دو ویژگی را ایجاد کند.

۶ ایده کلی پیاده‌سازی من

من برای سؤال دوم، کدم را طوری طراحی کردم که هسته اصلی DES یک بار نوشته شود و فقط S-boxها به‌صورت پارامتر به آن داده شوند. این کار دو تا مزیت داشت:

۱. منطق اصلی DES یک جا متمرکز می‌ماند و کد تکراری نمی‌شود.

۲. مقایسه بین DES استاندارد و نسخه تصادفی منصفانه‌تر می‌شود، چون تنها چیزی که عوض می‌شود خود S-boxها هستند.

یعنی عملاً من یک تابع رمزگذاری داشتم که می‌توانست با هر مجموعه S-box کار کند. بعد یک بار S-boxهای استاندارد را به آن دادم و یک بار S-boxهای تصادفی را.

۷ توضیح جزئی تر کد

۱۰۷ تعریف جدول‌های ثابت

در ابتدای کد، جدول‌های استاندارد DES را گذاشتم. این جدول‌ها شامل موارد زیر بودند:

- IP برای جایگشت اولیه
- IP_INV برای جایگشت نهایی
- EXPANSION برای تبدیل ۳۲ بیت به ۴۸ بیت
- P برای جایگشت داخل تابع F
- PC1 و PC2 برای ساخت کلیدهای دور
- SHIFTS برای میزان شیفت در هر دور
- STANDARD_SBOXES برای ۸ عدد S-box استاندارد

این بخش پایه DES است. اگر هر کدام از این جدول‌ها اشتباه باشند، کل خروجی‌ها از استاندارد فاصله می‌گیرند.

۲۰۷ تابع apply_table

برای اینکه اعمال جایگشت‌ها را در چند جای مختلف کد به صورت تکراری ننویسم، یک تابع عمومی تعریف کردم که یک BitVector و یک جدول جایگشت می‌گیرد و نتیجه جایگشت را برمی‌گرداند. چون جدول‌های DES معمولاً از ۱ شماره‌گذاری می‌شوند ولی اندیس‌گذاری پایتون از ۰ شروع می‌شود، در این تابع این اختلاف را هم مدیریت کردم.

۳۰۷ تبدیل متن به بیت

برای ورودی‌های ۸ کاراکتری، از BitVector استفاده کردم تا متن را به یک رشته بیتی ۶۴ بیتی تبدیل کنم. این کار باعث شد عملیات‌های بیتی مثل XOR، تقسیم به دو نیمه، و جایگشت راحت‌تر و خواناتر شوند.

۴۰۷ ساخت کلیدهای دور

در تابع ساخت کلیدهای دور، این مراحل را انجام دادم:

۱. کلید ۸ کاراکتری را به ۶۴ بیت تبدیل کردم.
۲. با PC1، آن را به ۵۶ بیت کاهش دادم.
۳. کلید ۵۶ بیتی را به دو بخش ۲۸ بیتی تقسیم کردم.

- ۴. در هر دور، هر دو بخش را طبق جدول SHIFTS به چپ چرخاندم.
 - ۵. دو بخش را دوباره به هم چسباندم.
 - ۶. با PC2 کلید ۴۸ بیتی همان دور را ساختم.
- در انتها یک آرایه از ۱۶ کلید دور داشتم که در رمزگذاری از آن‌ها استفاده شد.

۵.۷ تابع جانشانی با S-box

برای بخش جانشانی، یک تابع نوشتم که یک ورودی ۴۸ بیتی می‌گیرد و آن را با مجموعه S-boxهایی که به آن داده شده پردازش می‌کند. داخل این تابع:

- ورودی ۴۸ بیتی را به ۸ قطعه ۶ بیتی تقسیم کردم
 - برای هر قطعه، از بیت اول و آخر شماره سطر را ساختم
 - از چهار بیت وسط شماره ستون را ساختم
 - مقدار داخل خانه مربوطه از S-box را خواندم
 - آن مقدار را به ۴ بیت تبدیل کردم
 - در نهایت ۸ خروجی ۴ بیتی را کنار هم گذاشتم و ۳۲ بیت ساختم
- این قسمت خیلی مهم بود چون کوچک‌ترین اشتباه در انتخاب سطر و ستون، کل رفتار الگوریتم را خراب می‌کرد.

۶.۷ تابع feistel

این تابع، همان تابع F در DES است. در این تابع:

- ۱. نیمه راست ۳۲ بیتی را expand کردم
 - ۲. با کلید دور XOR کردم
 - ۳. از تابع S-box استفاده کردم
 - ۴. روی خروجی ۳۲ بیتی حاصل، جایگشت P را اعمال کردم
- خروجی این تابع همان چیزی بود که در هر دور با نیمه چپ XOR می‌شد.

۷.۷ تابع اصلی رمزگذاری بلوک

در تابع اصلی رمزگذاری، ابتدا جایگشت اولیه را روی بلوک ۶۴ بیتی اعمال کردم. بعد آن را به دو نیمه ۳۲ بیتی تقسیم کردم. سپس ۱۶ دور Feistel اجرا کردم. بعد از پایان ۱۶ دور، دو نیمه را با ترتیب نهایی DES به هم چسباندم و در انتها جایگشت معکوس را اعمال کردم.

این تابع عملاً هسته اصلی DES بود و چون S-box ها را به صورت پارامتری می گرفت، هم برای DES استاندارد استفاده شد و هم برای DES با S-box تصادفی.

۸.۷ ساخت S-box تصادفی

برای بخش تصادفی، تابعی نوشتم که ۸ جدول بسازد. هر جدول ۴ سطر و ۱۶ ستون دارد و هر خانه اش با یک عدد صحیح تصادفی بین ۰ تا ۱۵ پر می شود. به این ترتیب ساختار کلی S-box از نظر ابعاد درست می ماند، ولی محتوایش دیگر استاندارد نیست.

نکته ای که اینجا برایم مهم بود این بود که سؤال نگفته بود حتماً اعداد هر سطر باید یگنا باشند یا جدول ویژگی خاصی داشته باشد؛ فقط گفته بود خانه ها با اعداد صحیح درست و به صورت تصادفی پر شوند. برای همین تولید تصادفی ساده و مستقیم انجام دادم.

۹.۷ محاسبه اثر بهمنی

برای محاسبه اثر بهمنی، این روند را اجرا کردم:

۱. ابتدا ciphertext متن اصلی را حساب کردم.
 ۲. بعد به ترتیب هر بار یکی از ۶۴ بیت plaintext را برعکس کردم.
 ۳. برای هر plaintext تغییرکرده، دوباره رمزگذاری را انجام دادم.
 ۴. فاصله همینگ بین ciphertext اصلی و ciphertext جدید را حساب کردم.
 ۵. مجموع این فاصله ها را جمع کردم.
 ۶. در انتها بر ۶۴ تقسیم کردم تا میانگین تعداد بیت های تغییرکرده به دست بیاید.
- بعد برای اینکه نتیجه قابل فهم تر باشد، این مقدار را به درصد هم تبدیل کردم:

$$\text{Percent Avalanche} = \frac{\text{Bits Changed Average}}{64} \times 100$$

۸ جزئیات منطقی صحت سنجی

من برای اینکه مطمئن شوم کد درست کار می کند، فقط به یک اجرا اکتفا نکردم. چند نوع بررسی انجام دادم:

۱۰۸ پایداری DES استاندارد

برای DES استاندارد، اگر plaintext و key یکسان باشند، ciphertext باید همیشه یکسان باشد. در خروجی‌های من دقیقاً همین اتفاق افتاد. مثلاً برای ورودی UTCLASS! و کلید 87654321، در چند اجرای مختلف ciphertext استاندارد همیشه یکی بود. این نشان می‌دهد بخش استاندارد کاملاً deterministic کار می‌کند و جایی از کد وابستگی تصادفی ناخواسته وجود ندارد.

۲۰۸ تغییرپذیری نسخه تصادفی

برای نسخه‌ای که از S-box تصادفی استفاده می‌کند، طبیعی است که اگر هر بار S-boxها را از نو بسازم، ciphertext عوض شود. در اجراهای من این هم دقیقاً دیده شد. یعنی برای یک plaintext و key ثابت، ciphertext نسخه تصادفی در اجراهای مختلف متفاوت بود. این رفتار هم درست است و نشان می‌دهد تابع تولید S-box تصادفی واقعاً فعال بوده است.

۳۰۸ منطقی بودن اندازه اثر بهمی

در خروجی‌ها، مقدار میانگین بیت‌های تغییرکرده معمولاً نزدیک ۳۲ بود. چون خروجی ۶۴ بیت است، این یعنی حدود نصف بیت‌ها تغییر می‌کنند که دقیقاً همان چیزی است که از یک الگوریتم با اثر بهمی خوب انتظار داریم. اگر این مقدار خیلی پایین بود، مثلاً حوالی ۱۰ یا ۱۵، احتمال زیادی داشت که کد یا منطق الگوریتم مشکل داشته باشد.

۹ نتایج اجرای برنامه

من برنامه را با چند نمونه مختلف اجرا کردم. نتیجه‌ها به صورت زیر بودند.

۱۰۹ نمونه اول

ABCDEFGH plaintext:
12345678 key:

خروجی:

DES Standard
96de603eae6256f (hex): Ciphertext
69.31 bits: changed Average
51%.49 percent: Avalanche

-boxesS Random with DES
c9464c3deb884749 (hex): Ciphertext
95.31 bits: changed Average

تحلیل من از این اجرا این بود که هر دو نسخه اثر بهمی خوبی دارند، چون هر دو نزدیک ۳۲ بیت تغییر را نشان می‌دهند. در این اجرا نسخه تصادفی کمی عدد بالاتری داده، ولی اختلاف خیلی کم است.

۲.۹ نمونه دوم

HELLOWRD plaintext:
SECRETS! key:

خروجی:

DES Standard
5f3acde2a0c31b35 (hex): Ciphertext
70.31 bits: changed Average
54%.49 percent: Avalanche

-boxesS Random with DES
a9b12d9e98af9527 (hex): Ciphertext
53.32 bits: changed Average

در این نمونه هم DES استاندارد بسیار نزدیک به مقدار ایده‌آل بوده و نسخه تصادفی هم اتفاقاً کمی بیشتر از آن شده است. این نتیجه نشان می‌دهد یک S-box تصادفی هم ممکن است در یک اجرای خاص اثر بهمنی نسبتاً خوبی نشان دهد، ولی این به‌تنهایی کافی نیست که بگوییم از نسخه استاندارد بهتر است.

۳.۹ نمونه سوم

NETWORK! plaintext:
DESKEY12 key:

خروجی:

DES Standard
d230878515a394ca (hex): Ciphertext
77.31 bits: changed Average
63%.49 percent: Avalanche

-boxesS Random with DES
5b457808cc7c79fa (hex): Ciphertext
94.31 bits: changed Average

در این تست هم هر دو نسخه نزدیک ۳۲ بودند و باز نسخه تصادفی کمی بالاتر قرار گرفت. اما هنوز اختلاف در حدی نیست که بتوان از روی یک نمونه خاص قضاوت قطعی کرد.

۴.۹ نمونه چهارم

CRYPT012 plaintext:
SAFEKEY! key:

خروجی:

DES Standard
886380dd908b0a85 (hex): Ciphertext
09.32 bits: changed Average
15%.50 percent: Avalanche

-boxesS Random with DES
2d59b637536b7710 (hex): Ciphertext
08.32 bits: changed Average

اینجا تقریباً هر دو نسخه رفتار یکسانی داشته‌اند. DES استاندارد خیلی خیلی کم بهتر بوده. این نتیجه برای من جالب بود چون نشان می‌دهد از نظر شاخص ساده اثر بهمنی، بعضی S-box های تصادفی هم ممکن است به DES استاندارد نزدیک شوند.

۵.۹ نمونه پنجم

UTCLASS! plaintext:
87654321 key:

خروجی اجرا اول:

DES Standard
d6a5c5ec30d61c23 (hex): Ciphertext
83.31 bits: changed Average
73%.49 percent: Avalanche

-boxesS Random with DES
5b21f02e2c8f0caa (hex): Ciphertext
81.31 bits: changed Average

خروجی اجرا دوم با همان ورودی و کلید:

DES Standard
d6a5c5ec30d61c23 (hex): Ciphertext
83.31 bits: changed Average
73%.49 percent: Avalanche

-boxesS Random with DES
719375890f55c48d (hex): Ciphertext
30.32 bits: changed Average
46%.50 percent: Avalanche

خروجی اجرا سوم با همان ورودی و کلید:

DES Standard
d6a5c5ec30d61c23 (hex): Ciphertext
83.31 bits: changed Average

73%.49 percent: Avalanche

-boxesS Random with DES
f6a80a17c00f78cd (hex): Ciphertext
14.31 bits: changed Average
66%.48 percent: Avalanche

این سه اجرای پشت سرهم از همه جالب تر بودند، چون دقیقاً دو ویژگی مهم را نشان دادند:

- DES استاندارد برای ورودی و کلید ثابت، دقیقاً خروجی و اثر بهمنی ثابتی داد.
 - DES با S-box تصادفی، هر بار نتیجه متفاوتی داد؛ یک بار بهتر، یک بار تقریباً مشابه، و یک بار ضعیف تر.
- به نظرم این بهترین شاهد است برای اینکه نسخه تصادفی رفتار ثابتی ندارد و کیفیت آن به جدول های تصادفی تولید شده وابسته است.

۶.۹ نمونه ششم

SECURITY plaintext:
PASSKEY1 key:

خروجی:

DES Standard
2144b250394d7e7c (hex): Ciphertext
64.32 bits: changed Average
00%.51 percent: Avalanche

-boxesS Random with DES
06e0d268359eced (hex): Ciphertext
25.32 bits: changed Average

در این تست DES استاندارد از نسخه تصادفی بهتر ظاهر شد و حتی کمی بالاتر از ۵۰ درصد رفت. این نتیجه هم خیلی طبیعی است و با انتظار تئوری سازگار است.

۱۰ تحلیل نهایی نتایج

اگر بخواهم خروجی ها را جمع بندی کنم، چند نکته مهم دیده می شود.

۱۰.۱۰ DES استاندارد بسیار پایدار است

در همه تست ها، DES استاندارد تقریباً همیشه اثری نزدیک به ۳۲ بیت تغییر نشان داده است. این یعنی رفتار آن هم پایدار است و هم از نظر اثر بهمنی در محدوده مطلوب قرار دارد. این نتیجه با انتظار تئوری از DES سازگار است، چون S-box های آن تصادفی طراحی نشده اند، بلکه برای داشتن ویژگی های امنیتی مناسب انتخاب شده اند.

۲۰۱۰ S-box تصادفی الزاماً بد نیست، ولی قابل اتکا هم نیست

در بعضی اجراها نسخه تصادفی کمی بهتر از DES استاندارد عمل کرده و در بعضی اجراها ضعیف تر بوده. این موضوع نشان می‌دهد که صرفاً تصادفی بودن یک جدول لزوماً باعث خراب شدن فوری رفتار اثر بهمنی نمی‌شود. چون معیار ما اینجا فقط یک شاخص آماری ساده بوده، بعضی جدول‌های تصادفی ممکن است اتفاقاً خروجی قابل قبولی بدهند.

اما نکته اصلی اینجا است که این رفتار پایدار و مهندسی شده نیست. یعنی امروز یک S-box تصادفی ممکن است خوب جواب بدهد و فردا یکی دیگر بد جواب بدهد. DES استاندارد این نوسان را ندارد.

۳۰۱۰ اثر بهمنی خوب، به تنهایی معیار کامل امنیت نیست

یکی از نکات مهمی که از این سؤال برایم روشن تر شد این بود که نزدیک بودن به ۵۰ درصد تغییر در خروجی، شرط خوبی است ولی شرط کافی نیست. یعنی اگر یک الگوریتم یا یک S-box اثر بهمنی نسبتاً خوب نشان بدهد، هنوز نمی‌توان نتیجه گرفت که از نظر امنیتی طراحی خوبی دارد. معیارهای دیگری هم وجود دارند، مثل مقاومت در برابر تحلیل تفاضلی و خطی، توزیع خروجی‌ها، و ساختار کلی نگاشت غیرخطی. پس اگرچه S-box‌های تصادفی من در بعضی اجراها عدد‌های خوبی دادند، ولی این به معنی بهتر بودن آن‌ها از S-box‌های استاندارد نیست.

۱۱ صحت سنجی نهایی

من نتایج را از چند زاویه معتبر دیدم:

۱. ثابت بودن خروجی استاندارد برای ورودی ثابت: این نشان داد پیاده‌سازی DES استاندارد درست و بدون وابستگی تصادفی اجرا می‌شود.

۲. تغییر خروجی نسخه تصادفی بین اجراها: این نشان داد تولید S-box تصادفی واقعاً در حال انجام است.

۳. نزدیکی میانگین تغییر بیت‌ها به ۳۲: این نشان داد محاسبه اثر بهمنی به صورت منطقی انجام شده و خروجی‌ها در بازه مورد انتظار هستند.

۴. تغییر کامل $ciphertext$ با تغییر $plaintext$ و key : این هم از سلامت کلی پیاده‌سازی پشتیبانی می‌کند.

از روی این موارد، من نتیجه گرفتم که هم بخش DES استاندارد درست پیاده‌سازی شده و هم بخش تولید S-box تصادفی و محاسبه اثر بهمنی درست کار می‌کنند.

```
PS C:\Users\A\Desktop\HW1_starter_code> & C:/Users/a/anaconda3/2/python.exe c:/Users/a/Desktop/HW1_starter_code/q2_des.py
DES Avalanche Effect Test
-----
Enter plaintext (8 chars): UTCLASS!
Enter key (8 chars): 87654321

Standard DES
Ciphertext (hex): d6a5c5ec30d61c23
Average changed bits: 31.83
Avalanche percent: 49.73%

DES with Random S-boxes
Ciphertext (hex): f6a80a17c00f78cd
Average changed bits: 31.14
Avalanche percent: 48.66%

Comparison
Standard DES produced stronger avalanche effect in this run.
```

شکل ۱: نمونه‌ای از اجرای برنامه و محاسبه اثر بهمنی